

Traceprop

Breast-Cancer Demo — Code Walkthrough

A line-by-line trace of `hf_space/app.py` through the library

Generated for personal reference

1. What the demo is

The Hugging Face Space (`hf_space/app.py`) is a Gradio app with three interactive tabs, all backed by *one* small model and *one* shared data structure:

- **Dataset:** Wisconsin Breast Cancer — 569 samples, 30 numeric features, binary label (1 = benign, 0 = malignant).
- **Model:** plain `sklearn LogisticRegression` (C=10).
- **Shared object:** a `GradientStore` holding one compressed *gradient fingerprint* per training sample.

The whole point: a logistic-regression classifier is *linear*, so its per-sample last-layer gradient is available in closed form. That makes Traceprop's attribution **mathematically exact** here — the ideal showcase.

The three tabs answer three questions off that single store:

Tab	Question	Mechanism
Attribution	Which training patients drove this prediction?	dot / TRAK on fingerprints
Provenance	Where did this data come from?	lineage-graph walk
Unlearning	Make the model forget one patient.	reverse gradient steps

2. The big idea — how the whole library works

Before the demo trace, here is the architecture the demo sits on top of.

Traceprop answers three questions about a trained model, and it does so with *one* shared data structure: a store of compressed per-sample gradients. Everything — attribution, unlearning, and source-level provenance — is a linear-algebra operation on that store.

There are really **two independent subsystems** in the package that share a name:

1. **Lineage tracking** (`tensor.py`, `graph.py`, `interceptor.py`) — a NumPy wrapper that records *where data came from* (data provenance).
2. **Gradient attribution** (`attribution/`, `unlearning/`) — influence functions that record *which training samples drove a prediction*.

They connect through a `source_node_id` that ties each stored gradient back to a lineage-graph node. Here is each one.

2.1 Subsystem 1 — Lineage tracking (data provenance)

`ProvenanceTensor` (`tensor.py:26`) is a subclass of `np.ndarray`. It overrides NumPy’s dunder hooks — `__array_ufunc__` (for `+`, `*`, etc.) and `__array_function__` (for `np.concatenate`, etc.). Every time you do arithmetic on one:

- The real NumPy op runs (fast path at `tensor.py:54`).
- The interceptor records a *node* in a global `LineageGraph` with edges to its input tensors’ nodes.

So a computation graph is built as a *side effect* of ordinary NumPy code. Later you can call `.ancestors()` / `.sources()` on a result to walk back to the original CSV rows / tables it descends from — this is the < 1 ms “where did this data come from?” number. There is no training or gradients here; it is pure DAG bookkeeping. Granularity is tunable (`Granularity.OP` vs coarser) so you do not pay to track every scalar op.

2.2 Subsystem 2 — Gradient attribution (the core algorithm)

This is the influence-function machinery. Three stages.

Stage A — Log gradients during training. `TrainingContext.step()` (`training_context.py:67`) hooks your training loop. After each backward pass it (1) flattens *all* parameter gradients into one long vector g_i (per sample), and (2) hands it to `GradientStore.log_gradient`.

Stage B — Compress with a random projection. A full gradient vector can be millions of dimensions — untenable to store one per sample. So `RandomProjection` (`gradient_store.py:38`) compresses each g_i from `input_dim` $\rightarrow$ `proj_dim` (default 4096) via a sparse Johnson–Lindenstrauss projection:

$$R \in \{-1, 0, +1\}, \quad P = \{\frac{1}{6}, \frac{2}{3}, \frac{1}{6}\}, \quad \text{scaled by } \sqrt{3/\text{proj_dim}}, \quad \text{proj-}g_i = R g_i.$$

The JL lemma guarantees dot products (hence gradient *similarities*) are approximately preserved. Crucially the *same* seeded matrix is used for every sample — that is what makes the projected dot products comparable. `log_batch` (`gradient_store.py:120`) does a whole batch in one BLAS matmul (the $45\times \rightarrow 14\times$ speedup in the changelog).

Stage C — Attribute a test prediction. The influence of training sample z_i on a test point z_{test} is the classic influence-function formula (`influence.py:5`):

$$I(z_i, z_{\text{test}}) \approx -\nabla L(z_{\text{test}})^\top H^{-1} \nabla L(z_i).$$

Two estimators implement this:

1. **Raw dot product** (`compute_influence_scores`, `influence.py:26`) — drops H^{-1} :

$$\text{scores} = \Phi_{\text{train}} \varphi_{\text{test}} \quad (n \times d) \cdot (d) \rightarrow (n),$$

then L_∞ normalise. This is “Traceprop-LL (dot)”: fast, and exact for linear models / frozen-backbone linear probes, where it even beats TRAK.

2. **TRAK estimator** (`compute_trak_scores`, `influence.py:54`) — approximates H^{-1} with the regularised inverse Gram matrix in projected space:

$$G = \Phi^\top \Phi + \lambda I, \quad \text{scores} = \Phi_{\text{train}} (G^{-1} \varphi_{\text{test}}),$$

solved via Cholesky (`cho_factor/cho_solve`). The Gram inverse whitens correlated gradient directions — the Gauss–Newton/Fisher approximation to the Hessian. `precompute_gram_factor` caches the Cholesky so many test points are cheap. This lifts tabular LDS from 0.62 → 0.88.

`top_k_influential` (`influence.py:283`) then sorts and returns the top samples — and because each entry carries `source_node_id`, it attaches a `ProvenanceView` so you can jump straight from “influential sample #4172” back into the lineage graph.

The genuinely novel bit — source-level attribution. `compute_source_stratified_scores` (`influence.py:164`) aggregates per-sample scores up to the *source file / pipeline stage* by walking the lineage graph (`resolve_source_key`, `influence.py:266`). TRAK / LogIX / dattri treat training samples as opaque indices; this bridges the two subsystems to answer “which source table drove this prediction?” — the stated research contribution.

2.3 Unlearning (the library version)

`gradient_correction_unlearn` (`unlearning/gradient_correction.py:66`) reuses the same store:

1. Find all stored gradients whose `source_id` matches the data to forget.
2. Take their *mean projected gradient*.
3. Apply gradient-ascent-style correction: subtract $\text{lr} \cdot \text{mean_grad}$ from those rows, decaying by 0.95 each step, to drive their influence toward zero.
4. Verify by recomputing influence and checking it dropped below a threshold.

It is honest about being *approximate* (`compliance_report` at `gradient_correction.py:44` literally disclaims formal privacy guarantees vs. retraining). The benchmarks measure it against gold-standard retraining via “gap closed.”

2.4 Why it is fast

The whole thing is: project gradients once → store an $(n \times 4096)$ float32 matrix → attribution is one matmul (dot) or one Cholesky solve (TRAK). No Hessian is ever materialised; no GPU is required for tabular / linear cases. That is the source of the “0.22 s CPU, 266× faster than TRAK” claims.

Where the demo fits. The breast-cancer demo exercises *both* subsystems: Tabs 1 & 3 use Subsystem 2 (gradient attribution + unlearning) on a logistic-regression model; Tab 2 uses Subsystem 1 (lineage tracking) on two synthetic hospital tables. The rest of this document traces that demo end to end.

3. Startup: building the shared state

Everything is computed *once* at launch inside `_build_state()` (`app.py:53`). Here is the trace, step by step.

3.1 Load, split, scale, fit

```
data = load_breast_cancer() # 569 x 30, y: 1=benign 0=malignant
X_raw = data.data.astype(np.float32)
y_raw = data.target.astype(float)

rng = np.random.RandomState(42) # fixed seed -> reproducible demo
idx = rng.permutation(len(X_raw))
tr, te = idx[:400], idx[400:] # 400 train / 169 test

scaler = StandardScaler().fit(X_raw[tr]) # fit on TRAIN only
X_tr = scaler.transform(X_raw[tr]) # (400, 30)
X_te = scaler.transform(X_raw[te]) # (169, 30)

clf = LogisticRegression(C=10, solver="lbfgs", max_iter=500).fit(X_tr, y_tr)
coef = clf.coef_[0] # w : shape (30,)
intercept = clf.intercept_[0] # b : scalar
```

After this we have trained weights $w \in \mathbb{R}^{30}$ and bias b . The model's prediction for a feature vector x is

$$p(x) = \sigma(w^\top x + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

3.2 The gradient fingerprint — the heart of everything

For every training sample the demo computes a *last-layer gradient* and logs it (`app.py:74`):

```
store = GradientStore(proj_dim=512, seed=42)
for i in range(len(X_tr)):
    g = _per_sample_ll_grad(X_tr[i], y_tr[i], coef, intercept) # (30,)
    store.log_gradient(g, sample_index=i, source_id="breast_cancer_train")
```

where `_per_sample_ll_grad` (`app.py:45`) is:

```
def _per_sample_ll_grad(x_i, y_i, coef, intercept):
    logit = x_i @ coef + intercept
    err = sigmoid(logit) - y_i # residual = predicted - actual
    return (err * x_i) # gradient of BCE loss w.r.t. weights
```

Why this is exactly the gradient. The binary cross-entropy loss for one sample is $\ell_i = -y_i \log p_i - (1 - y_i) \log(1 - p_i)$ with $p_i = \sigma(w^\top x_i + b)$. Differentiating w.r.t. the weights gives the classic logistic-regression result:

$$\nabla_w \ell_i = (\sigma(w^\top x_i + b) - y_i) x_i = \underbrace{(p_i - y_i)}_{\text{err}} x_i.$$

So `err * x_i` is the true gradient — no approximation. This vector is the sample’s “fingerprint”: a 30-dim record of *how* sample *i* tugged the weights.

3.3 What `log_gradient` does with it

Inside `GradientStore.log_gradient` ([gradient_store.py:88](#)) the first call lazily builds a random projection, then compresses and stores the fingerprint:

```
if self._projection is None: # built once, on first sample
    self._projection = RandomProjection(input_dim=30, proj_dim=512, seed=42)
proj_grad = self._projection.project(flat) # 30 -> 512
entry = GradientLogEntry(proj_gradient=proj_grad, source_id=..., sample_index=i)
self._entries[entry.id] = entry
```

The projection is a **sparse Johnson–Lindenstrauss** matrix ([gradient_store.py:38](#)): entries drawn from $\{-1, 0, +1\}$ with probabilities $\{\frac{1}{6}, \frac{2}{3}, \frac{1}{6}\}$, scaled by $\sqrt{3/\text{proj_dim}}$. Its defining property is that it *preserves dot products in expectation*:

$$\mathbb{E}[\langle Rg_a, Rg_b \rangle] = \langle g_a, g_b \rangle,$$

which is exactly what attribution needs, since attribution scores *are* dot products of gradients. The same seeded matrix R is reused for every sample so all fingerprints live in one comparable space.

Aside (honest detail): here `proj_dim=512` is *larger* than the 30-dim gradient, so this “projection” actually up-samples. That adds no compression benefit (a real deployment compresses million-dim gradients *down*), but it is harmless: with 30-dim inputs the dot products are essentially preserved exactly. For this tiny model you could skip projection entirely; it is kept so the demo path matches the real library path.

After the loop we hold a **(400, 512) gradient matrix** Φ — one row per training patient. Finally two attribution engines are built ([app.py:99](#)):

```
engine_dot = AttributionEngine(store, estimator="dot")
engine_trak = AttributionEngine(store, estimator="trak", lambda_factor=1e-3)
```

Constructing the TRAK engine eagerly factorises the Gram matrix once ([attribution_engine.py:92](#)) so every later query is cheap — see §4.4.

4. Tab 1 — Attribution trace

When you move the sliders and hit *Run*, Gradio calls `run_attribution(test_idx, estimator, top_k)` ([app.py:115](#)).

4.1 Step 1 — predict on the test patient

```
x = X_te[test_idx]; y = y_te[test_idx]
logit = x @ coef + intercept
prob = sigmoid(logit); pred = int(prob >= 0.5)
```

4.2 Step 2 — fingerprint the test patient the SAME way

```
err = sigmoid(logit) - y
test_grad = err * x # identical formula to training fingerprints
```

This is the key symmetry: the test query is turned into a gradient with the exact same $(p - y)x$ formula, so it lives in the same space as the stored training fingerprints.

4.3 Step 3 — score against all 400 training fingerprints

```
engine = engine_trak if estimator == "TRAK" else engine_dot
result = engine.attribute(test_grad, top_k=top_k)
entries = result.top(top_k)
```

Inside `AttributionEngine.attribute` (`attribution.engine.py:150`):

1. Project the test gradient: $\varphi_{\text{test}} = R g_{\text{test}} \in \mathbb{R}^{512}$.
2. Compute a score for every training row (dot or TRAK, below).
3. `top_k_influential` (`influence.py:283`) argsorts the scores, attaches each entry's `sample_index`, `source_id` and `source_node_id`, and returns the ranked list.

4.4 The two estimators

Dot product (`compute_influence_scores`, `influence.py:26`) — the raw gradient-similarity score:

$$\text{scores} = \Phi \varphi_{\text{test}} \quad (400 \times 512) \cdot (512) \rightarrow (400),$$

then divided by its max absolute value (L_∞ normalise). One matmul, sub-millisecond.

TRAK (`compute_trak_scores`, `influence.py:54`) — inserts a regularised inverse-Gram factor that approximates the inverse Hessian (Gauss–Newton / Fisher), whitening correlated gradient directions:

$$G = \Phi^\top \Phi + \lambda I, \quad \text{scores} = \Phi (G^{-1} \varphi_{\text{test}}).$$

G is factorised once with a Cholesky decomposition (`scipy.linalg.cho_factor`) at engine-construction time, and every query is a cheap `cho_solve`. On tabular data TRAK lifts LDS from ≈ 0.62 to ≈ 0.88 ; on this already well-conditioned linear model the plain dot product is often just as good.

4.5 Step 4 — render

The top- K entries become (`app.py:136`) a ranked table and a horizontal bar chart. Bars are coloured by the training patient's *true class* (red = malignant, green = benign), and the header reports the attribution latency in milliseconds. That millisecond number — no retraining, no GPU — is the headline of the whole project.

5. Tab 3 — Unlearning trace

`run_unlearning(target_idx, n_steps, lr_exp)` (app.py:307) makes the model “forget” one training patient by **gradient ascent** on that sample’s loss:

```
coef_new, intercept_new = coef.copy(), intercept
for _ in range(n_steps):
    logit = x_forget @ coef_new + intercept_new
    err = sigmoid(logit) - y_forget
    coef_new += lr * err * x_forget # NOTE: += is ASCENT (undo the tug)
    intercept_new += lr * err
```

During training, sample i pulled the weights along $-\nabla_w \ell_i$ (descent). Adding a few steps of $+\nabla_w \ell_i$ ($+=$) pushes the weights *back*, raising the loss on that one patient while barely moving the other 399. The tab shows two bars:

- **Left:** cross-entropy loss on the forgotten patient — jumps *up* (the model no longer “knows” it).
- **Right:** overall test accuracy — stays essentially flat (everyone else is preserved). The gap between the two is the entire value proposition.

Honest detail: this tab implements the reverse-gradient idea *inline* on the sklearn weights for clarity. The library’s production version, `unlearning/gradient_correction.py` → `gradient_correction_unlearn()`, does the same thing at scale: it selects target samples by `source_id` in the `GradientStore`, uses their mean projected gradient, applies a decaying correction, then *re-verifies* that influence dropped below a threshold — and ships an EU-AI-Act audit record. Same idea, more plumbing.

6. Tab 2 — Provenance trace (brief)

`run_provenance` (app.py:184) is the *other* subsystem — pure data lineage, no gradients. It wraps two “hospital” arrays as `ProvenanceTensors` and does ordinary NumPy arithmetic on them:

```
source_a = tp.from_numpy(rows_a, source_id="hospital_A") # tracked root
source_b = tp.from_numpy(rows_b, source_id="hospital_B")
scaled_a = source_a * 0.5 # each op -> a graph edge
result = (scaled_a - mean) / (std + 1e-8) # StandardNorm, tracked
view = tp.provenance(result)
view.ops(); view.ancestors() # walk the lineage DAG
```

`ProvenanceTensor` (`tensor.py:26`) subclasses `np.ndarray` and overrides `__array_ufunc__` / `__array_function__`, so *every* arithmetic op silently records a node + edge in a global lineage graph. Tracing a value back to its source hospitals is then just a graph walk — the sub-millisecond “where did this come from?” number. This graph is what links back to attribution: each stored gradient carries a `source_node_id` into this same graph, enabling source-file-level attribution.

7. A complete worked example (real numbers)

To make it concrete, here is one full attribution query run through the actual demo code — **Test Patient #0**. These are real outputs, not illustrations.

7.1 The prediction

True label	malignant
Predicted	malignant (correct)
$P(\text{benign})$	0.0001 (logit $w^\top x + b = -8.99$)

The model is extremely confident this patient is malignant. We now ask: *which training patients pushed it there?* The test fingerprint is $g_{\text{test}} = (\sigma(-8.99) - 0) x = (0.0001) x$.

7.2 Dot-product estimator — top 5

Rank	Train #	Score	Label
1	307	−1.0000	malignant
2	248	−0.3390	malignant
3	82	−0.2193	malignant
4	138	−0.1661	benign
5	250	−0.1597	benign

Query latency: **0.78 ms**

Top-3 are all *malignant* — same class as the test patient.

Rank 1 is L_∞ -normalised to −1.0000 (largest magnitude).

Reading it. The strongest influencer, train#307, is itself a malignant patient whose gradient fingerprint aligns most strongly with the test patient’s — exactly the intuition “patients who look like this one, in the same class, drove the answer.” The raw influence definition ([influence.py:5](#)) carries a leading minus sign, so the strongest loss-reducing influencers surface as the largest-magnitude *negative* scores; [top_k_influential](#)’s default sort ([influence.py:298](#)) returns them first, and L_∞ normalisation pins the top one to ± 1 .

7.3 TRAK estimator — top 5 (same query)

Rank	Train #	Score	Label
1	395	−0.3555	benign
2	92	−0.1993	benign
3	108	−0.1841	benign
4	173	−0.1696	benign
5	176	−0.1140	malignant

Query latency: **4.20 ms**

The G^{-1} whitening re-weights the ranking: it downweights the crowded malignant cluster and surfaces *decision-boundary* patients instead.

TRAK’s inverse-Gram factor G^{-1} divides out directions that many training gradients share. On this extreme, easily-classified point it reshuffles the ranking toward boundary-adjacent samples rather than the nearest same-class cluster — illustrating why dot-product and TRAK can legitimately disagree, and why the demo lets you toggle between them.

7.4 The intuition, on a 4-row toy

The same machinery on a 2-feature, 4-row toy (this is the table in the demo’s “*How it works*” tab) makes the mechanism visible by hand:

Row	Features	Label	Fingerprint (sketch)
0	[0.9, 0.1]	benign	[+0.41, -0.12, ...]
1	[0.2, 0.8]	malignant	[-0.38, +0.45, ...]
2	[0.7, 0.3]	benign	[+0.39, -0.10, ...]
3	[0.1, 0.9]	malignant	[-0.42, +0.48, ...]

A test query with features [0.8, 0.2] has fingerprint [+0.40, -0.11, ...]. Dot-producting it against each row gives large positive alignment with rows 0 and 2 (the benign patients it resembles) and negative alignment with rows 1 and 3 — so the top-2 influential rows are #0 and #2. That is exactly what the real 30-feature / 400-patient run above does, just at scale.

8. End-to-end summary

The single pipeline, once at startup	
1	Train logistic regression on 400 patients.
2	For each patient: fingerprint $g_i = (p_i - y_i) x_i$ (exact BCE gradient).
3	Project 30 $\rightarrow$ 512 via seeded sparse JL; store rows $\Rightarrow \Phi \in \mathbb{R}^{400 \times 512}$.
4	Pre-factorise $G = \Phi^\top \Phi + \lambda I$ for the TRAK engine.
Per query (milliseconds, CPU)	
Attribution	fingerprint test point $\rightarrow \Phi \varphi$ (dot) or $\Phi G^{-1} \varphi$ (TRAK) $\rightarrow$ top- K .
Unlearning	take $+\nabla_w \ell$ steps on one patient; loss $\uparrow$, accuracy flat.
Provenance	walk the lineage DAG from output back to source hospitals.

One fingerprint per training sample answers all three questions — “who was influential?”, “where did this come from?”, “can you forget this?” — in milliseconds, on CPU, exactly, for this linear model.

9. Appendix: the same math, for a high-school reader

If the formulas above felt dense, here is every step rebuilt with small numbers you can check on paper. No calculus needed — just multiply and add.

9.1 Step 1 — turning a score into a probability (the sigmoid)

The model computes one number, the *logit*: multiply each feature by its weight, add them up, add the bias. Say it gets $z = 2$. We need a probability between 0 and 1, so we squash z with the **sigmoid**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma(2) = \frac{1}{1 + e^{-2}} = \frac{1}{1 + 0.135} \approx 0.88.$$

Think of it as a smooth on/off switch: big positive $z \rightarrow$ near 1 (“benign”), big negative $z \rightarrow$ near 0 (“malignant”), $z = 0 \rightarrow$ exactly 0.5 (a coin flip).

9.2 Step 2 — the “fingerprint” is just error $\times$ features

Each patient has features, e.g. $x = [2, 5]$, and a true label y (1 = benign, 0 = malignant). The model predicts a probability p . The **error** is simply

$$\text{err} = p - y \quad (\text{how wrong we were, with a sign}).$$

The fingerprint (the “gradient”) is that error multiplied through the features:

$$g = \text{err} \times x.$$

Concrete: if the true label is $y = 1$ but the model only says $p = 0.6$, then $\text{err} = 0.6 - 1 = -0.4$, and

$$g = -0.4 \times [2, 5] = [-0.8, -2.0].$$

That is it. **Intuition:** the sign of the error says which way to nudge the weights, and multiplying by the features says *which* weights to nudge and by how much (bigger feature $\Rightarrow$ bigger nudge). Every training patient gets one such list of numbers.

9.3 Step 3 — comparing two fingerprints (the dot product)

To measure how much two fingerprints *agree*, use the **dot product**: multiply matching entries and add them up. For $a = [3, -1]$ and $b = [2, 4]$:

$$a \cdot b = (3)(2) + (-1)(4) = 6 - 4 = 2.$$

- Big *positive* $\Rightarrow$ the two lists point the same way (very similar).
- Near *zero* $\Rightarrow$ unrelated.
- Big *negative* $\Rightarrow$ they point in opposite directions.

Attribution in one sentence: take the test patient’s fingerprint, dot-product it against *every* training patient’s fingerprint, and the biggest-magnitude scores are the training patients who most shaped the answer. Sorting 400 numbers — that is the whole “AI explainability” step.

9.4 Step 4 — shrinking long lists without losing the comparison (projection)

Real fingerprints can be millions of numbers long — too big to store one per patient. So we *shrink* each list to a fixed shorter length by multiplying it by a fixed grid of -1 , 0 , and $+1$ ’s (chosen once, at random). The near-magic fact (the Johnson–Lindenstrauss lemma) is that **dot products barely change** after shrinking: if two lists agreed before, their shrunk versions still agree by about the same amount. Analogy: a person’s *shadow* is lower-dimensional than the person, but two people who are similar still cast similar shadows. As long as everyone is measured against the *same* light (the same random grid), the comparisons stay fair.

9.5 Step 5 — TRAK: grading on a curve

Plain dot products can be fooled: if lots of patients share one common direction, that direction gets over-counted and drowns out the rest. **TRAK** fixes this by “grading on a curve” — it measures how common each direction is across all patients and divides it out, so a match on a *rare, distinctive* direction counts for more than a match on a crowded, common one. Same comparison as before, just fairer. (Under the hood this is the G^{-1} factor; you do not need it to get the idea.)

9.6 Step 6 — unlearning: undo the nudge

During training, patient #5 nudged the weights in some direction. To make the model *forget* patient #5, we nudge the weights a little bit in the *opposite* direction — literally add back that patient’s fingerprint a few times:

$$w_{\text{new}} = w + \text{lr} \times g_5, \quad \text{repeat a few steps.}$$

Here lr (“learning rate”) is a small step size like 0.01. After a few steps the model is confused specifically about patient #5 (its loss shoots up), while the other 399 patients still hold the weights in place, so overall accuracy barely moves. Like gently un-bending one wire in a bundle without disturbing the rest.

9.7 Step 7 — provenance: keep the receipts

This part has no gradients at all. Every time you add, multiply, or normalise a data array, Traceprop quietly writes down “this array came from those arrays” — like keeping every receipt and building a family tree. Later, to answer “where did this final number come from?”, you just follow the tree backwards to the original hospital tables. Following a tree is fast, which is why it takes under a millisecond.

The one-sentence version: give every training example a short list of numbers (its fingerprint), and then *everything* — who was influential, what to forget, where data came from — becomes “multiply lists and add,” which a computer does in milliseconds.

10. In plain English — no math, no code

Everything above assumed you were comfortable with weights, gradients, and dot products. This section assumes none of that. If you handed this document to a hospital administrator, a lawyer, or a curious relative, this is the part they would read. There are no formulas here — only pictures in words.

10.1 The problem it solves

Imagine a hospital uses an AI to help flag which patients’ tumours look malignant. The AI learned by studying thousands of past patient records. One day it flags a new patient, and someone — the doctor, the patient, a regulator — asks three perfectly reasonable questions:

- “*Why* did the AI say that? Which past patients led it to this conclusion?”
- “*Where* did the data behind this even come from? Which hospital, which file?”
- “This patient asked us to delete their record. Can we make the AI truly *forget* them — without retraining the whole thing from scratch?”

Normally these are very hard to answer. The AI is a black box: it gives an answer but no receipt. Retraining it from scratch to remove one person can take hours or days. Traceprop is a tool that answers all three questions in the time it takes to blink — and it does it by being organised, not by being clever.

10.2 The one trick: give everyone a fingerprint

Here is the whole idea in one image. When the AI studies a patient’s record during learning, that record “nudges” the AI’s opinion in some direction — a little toward “benign” or “malignant,” based on that patient’s specific measurements. Traceprop writes down that nudge as a short list of numbers and keeps it. Think of it as a **fingerprint**: a compact summary of *how each past patient shaped the AI’s thinking*.

Now the hospital has a filing cabinet with one fingerprint per past patient. That single cabinet is all Traceprop needs to answer every question. Nothing else gets stored, and the AI itself is never re-run.

10.3 Answering the three questions

“Which patients drove this decision?” (attribution). Take the new patient and make *their* fingerprint the same way. Then simply flip through the cabinet and ask, for each past patient, “how similar is your fingerprint to this new one?” The most similar fingerprints are, almost by definition, the past patients who most shaped the answer — usually people with a similar diagnosis and similar measurements. Sorting a few hundred similarity scores is something a laptop does instantly. That is the entire “explain the AI” step: *find the fingerprints that look most alike*.

“Where did this data come from?” (provenance). This part has nothing to do with the AI at all. Every time the raw data is cleaned, combined, or adjusted on its way in, Traceprop quietly keeps a *receipt* — “this number came from those numbers, which came from that hospital’s spreadsheet.” Stack up all the receipts and you get a family tree for every value in the system. Answering “where did this come from?” is just following the tree backwards to the original source. Following a tree is fast, so the answer comes back in under a thousandth of a second.

“Can you make it forget this patient?” (unlearning). Remember that each patient nudged the AI’s opinion in some direction while it was learning. To make the AI forget one patient, Traceprop gives the AI a gentle nudge in the *opposite* direction — just enough to undo that one person’s influence. Afterwards the AI genuinely no longer “knows” that patient, yet everyone else is untouched, so its overall accuracy barely changes. It is like carefully un-bending a single wire in a bundle without disturbing the others — far cheaper than melting the whole bundle down and starting over (which is what retraining from scratch would mean).

10.4 Why anyone should care

- **It is honest.** The AI can now show its work — “these five past cases led me here” — instead of being an unaccountable black box. That matters in medicine, lending, and anywhere a wrong decision has consequences.
- **It is fast and cheap.** All three answers come from one small filing cabinet of fingerprints, using only ordinary arithmetic. No expensive graphics chips, no hours of waiting — an ordinary computer does it in milliseconds.
- **It respects the right to be forgotten.** When someone asks to have their data removed, the system can actually remove their influence and *prove* it did — the kind of thing new privacy laws increasingly demand.

The whole thing, in one breath: give every past record a little numerical fingerprint when the AI learns it, and keep a receipt for where each piece of data came from. After that, “who influenced this decision,” “where did this data come from,” and “please forget this person” all turn into quick, cheap lookups — no black box, no retraining, no waiting.